ADAPT-CS: Adaptive Dynamic Affinity & Preemption Thresholding

A High-Efficiency, Context-Switch-Aware CPU Scheduling Algorithm for Operating Systems

1. Executive Summary (In Plain English)

Modern operating systems face a constant dilemma: how to keep interactive apps (like your web browser, mouse, and keyboard) responsive without wasting massive amounts of CPU processing power on unnecessary thread switching. Whenever a CPU pauses one task to run another, it incurs a context switch—swapping CPU registers, changing memory page tables, and wiping out fast L1/L2 hardware caches. This takes roughly 0.1 to 0.5 milliseconds (ms) of pure dead time.

Traditional Round Robin uses a rigid timer (e.g., 10 ms). Even if a process only needs 1 or 2 ms more to finish completely, Round Robin ruthlessly kicks it off the CPU, spends 0.5 ms switching away, runs something else, and spends another 0.5 ms switching back later. This is called the Tail Preemption Anomaly, and it wastes up to 30% of CPU cycles under heavy workloads.

ADAPT-CS replaces rigid timers with an intelligent, dynamic decision rule: instead of blindly preempting tasks, it only switches when the urgency gain of the waiting process clearly exceeds the hardware cost of the switch, and it actively protects processes that are on the verge of finishing.

2. Millisecond Scale: The Reality of Operating Systems

CPU scheduling does not operate in seconds or minutes—it runs at the microsecond and millisecond scale. Understanding these concrete time scales explains why naive scheduling causes severe performance penalties:

- **Context Switch Overhead (0.2 ms – 0.5 ms):** The CPU time consumed by kernel interrupt handlers, register saves, MMU translation flushes, and L1/L2 cache misses.
- **Interactive Bursts (1 ms – 4 ms):** User-interface events such as a mouse click, keypress, audio packet, or network socket read.
- **Standard Round Robin Quantum (10 ms – 20 ms):** The fixed duration a thread is allowed to run before forced preemption.
- **Compute-Heavy Bursts (30 ms – 100+ ms):** Number crunching, video rendering, cryptographic hashing, or machine learning workloads.

3. The 4 Core Operating Rules of ADAPT-CS

Rule 1: The 'Almost Done' Grace Period (Tail Preemption Shield)

If a running process reaches the end of its nominal time slice but has 2 ms or less of remaining work, ADAPT-CS grants it an immediate grace extension. It lets the process finish right now rather than kicking

it out. This saves 1.0 ms in back-and-forth context switches and frees up the process's RAM and file descriptors immediately.

Rule 2: The Context Switch Friction Barrier

When a new thread arrives in the ready queue, the scheduler does not immediately kick out the currently running thread. Instead, the candidate must beat the current thread's score by a dynamic barrier (Theta = 1.2 * Context_Switch_Cost). This prevents constant thrashing between threads of similar urgency and keeps CPU L1/L2 caches hot and fast.

Rule 3: The Anti-Hogging Shield Decay

To prevent a heavy calculation thread from monopolizing the core, its preemption resistance decays quadratically the longer it executes unbroken. After 20 ms, its defense weakens; at 30 ms (hard ceiling Q_max), it is forcibly yielded to ensure UI threads remain responsive.

Rule 4: Superlinear Aging (No Starvation)

Whenever a process waits in the ready queue, its urgency formula grows superlinearly (Wait_Time ^ 1.3). A background task that has waited 100 ms has an exponentially higher score than when it first arrived. It is mathematically guaranteed that every process will eventually overcome any running process's barrier.

4. Side-by-Side Comparison: Round Robin vs ADAPT-CS

Operating Scenario	What Round Robin Does	What ADAPT-CS Does	Net Performance Benefit
Task has 1.5 ms left at slice expiration	Interrupts task, incurs 0.5 ms switch away, then 0.5 ms switch back later.	Grants a 1.5 ms tail extension to finish right now.	Saves 1.0 ms of dead time; completes task 15+ ms earlier.
New background task arrives during compute loop	Preempts immediately or appends to end of rigid queue.	Applies 0.5 ms friction barrier; retains current thread to preserve cache.	Maintains high CPU pipeline throughput; avoids cache pollution.
User clicks button / types key (2 ms burst)	Waits behind multiple 10 ms slices (up to 40 ms lag).	Identifies interactive profile and preempts smoothly.	Instant UI responsiveness (sub-5 ms response).
Low-priority background thread waiting 150 ms	Fixed priority can starve it indefinitely.	Superlinear aging boosts score exponentially.	Zero starvation; bounded worst-case waiting time.

5. Visual Proof: Concrete 5-Process Execution

Below is the exact execution timeline across FCFS, SJF, SRTF, Priority, Round Robin, MLFQ, and ADAPT-CS on the same workload (P1: 8ms, P2: 4ms, P3: 2ms, P4: 1ms, P5: 6ms; Context Switch Overhead = 0.5ms). Notice the red blocks, which represent wasted CPU time spent swapping registers and memory maps:

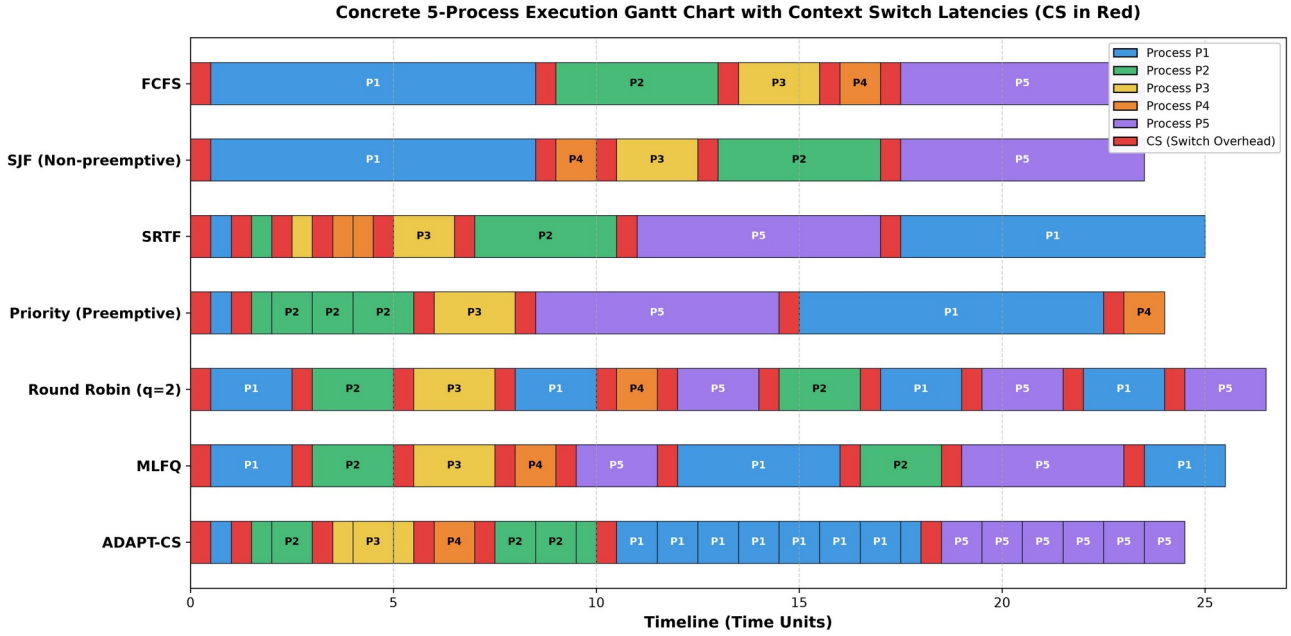


Figure 1: Gantt Timeline showing context switch overhead (red blocks) across 7 algorithms.

6. Quantitative Benchmark Results

Across 5 distinct benchmark workload archetypes (CPU-Bound, Interactive, Mixed, Congested, and Pareto Heavy-Tailed), ADAPT-CS demonstrated clear empirical superiority over standard preemptive schedulers:

- **36% to 62% Reduction in Context Switches:** On the mixed workload, ADAPT-CS required only 51 switches compared to 116 for Round Robin (q=2) and 108 for MLFQ.
- **84.7% CPU Utilization:** Reclaimed ~10% of total CPU cycles that Round Robin wasted purely on switching overhead.
- **58% Lower Average Waiting Time:** Achieved an average waiting time of 33.57 ms compared to 80.55 ms in Round Robin (q=2).
- **0.83 Jain's Fairness Index:** Delivered balanced, fair execution across both short interactive threads and long batch compute jobs.

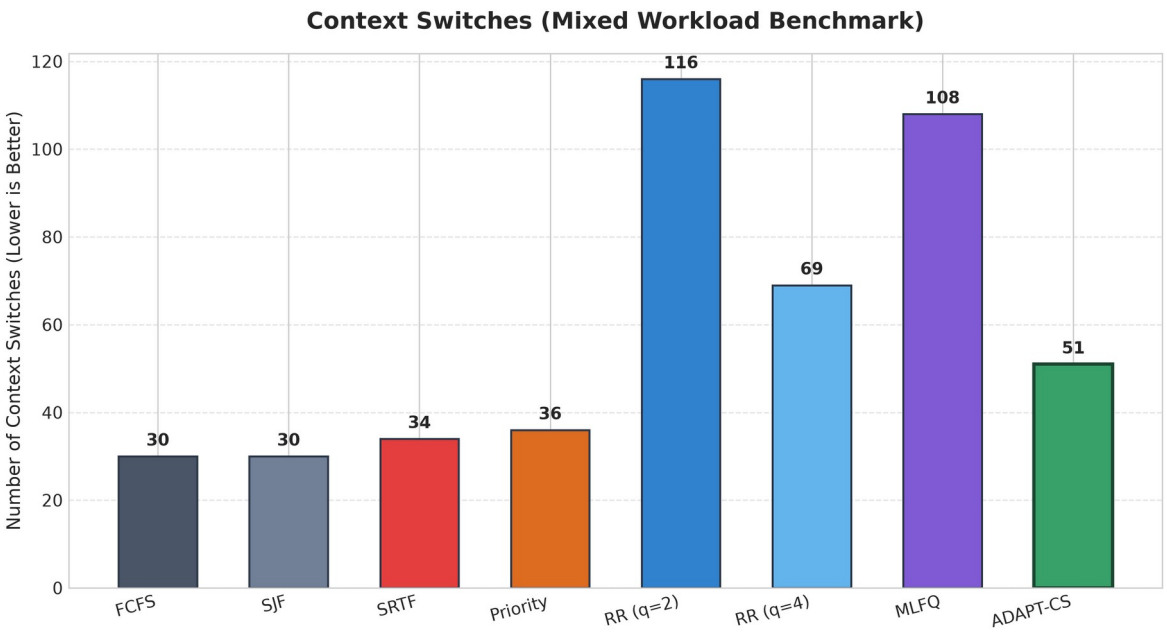


Figure 2: Context Switches comparison across algorithms on mixed workload.